

PP2 Report

Matchmaking Queue System

1. Group Member Names

- Phu Vinh Khang Dang
- Anh Tuan Huynh
- Hoang Minh Tam Dang

2. Project Name

Multiplayer Game Matchmaking Queue System

3. Instructions to Run the Program

1. Open a terminal and navigate to the project folder.
2. Compile the program with:

```
g++ -std=c++17 -Wall -Wextra -o matchmaker main.cpp Player.cpp Match.cpp RankedMatch.cpp
```

3. Run the program:

```
./matchmaker
```

4. Instructions for Using the Program

When the program starts, it asks for a Team Size (players per match, e.g. 2 for 1v1) and a Max MMR Gap (largest skill difference allowed, e.g. 200). Then a menu appears:

1. Join Normal Match - Adds a player to the casual queue. Enter a name, MMR (skill rating), and rank (Bronze-Diamond).
2. Join Ranked Match - Adds a player to the competitive queue. Same inputs as normal.
3. View Queues - Shows all players waiting in both the Normal and Ranked queues.
4. Find Matches - Runs the matchmaker on both queues. Similar-skill players are grouped. Ranked matches also pick a winner.
5. View Match History - Shows all completed matches and results.
6. Exit - Ends the program.

Adding to the queue: Select option 1 or 2, then follow the prompts. Removing from the queue: Select option 4. The matchmaker dequeues all players, sorts them by MMR, and groups them. Unmatched players are re-enqueued automatically.

5. Program Description

This program simulates how online multiplayer games match players into fair, balanced games. Players join either a Normal (casual) or Ranked (competitive) queue with a name, skill rating (MMR), and rank tier (Bronze through Diamond). The matchmaker then groups players with similar skill levels together, ensuring no match contains players whose skill gap exceeds a configurable threshold.

The system uses two separate queues - one for casual play and one for competitive play. The key difference is that Ranked matches also determine a winner (the highest-MMR player), while Normal matches are just for fun. This design mirrors how real games like League of Legends and Valorant separate casual and ranked play.

6. Implementation Explanation

Queue (Circularly Linked List)

The queue is implemented in Queue.h as a templated class using a circularly linked list. The inner Node struct (line 14) stores data and a next pointer. The queue maintains a m_tail pointer (line 22) - the last node in the list - and m_tail->next always points back to the front node, forming a circle.

enqueue() (Queue.h, line 68) creates a new node and inserts it after the tail in O(1). dequeue() (Queue.h, line 85) removes the front node and updates the circular link in O(1). The queue also implements a copy constructor (line 37) and copy assignment operator (line 49) for proper deep copying (rule of three).

The Matchmaker class (Matchmaker.h, line 20) uses Queue<shared_ptr<Player>> as its waiting queue. When forming matches, it calls dequeue() to drain all players into a vector (Matchmaker.cpp, line 24), sorts them by MMR, and groups them using a sliding window algorithm (Matchmaker.cpp, lines 64-86). Unmatched players are re-enqueued (Matchmaker.cpp, line 32).

Inheritance (Match -> RankedMatch)

The program uses inheritance between the Match base class and the RankedMatch derived class. Match (Match.h, line 11) stores a match ID, a vector of players, and average MMR. It has a virtual display() method (Match.h, line 30) that prints match info.

RankedMatch (RankedMatch.h, line 9) inherits from Match and adds a m_winnerIndex field (line 11). It overrides display() (line 24) to also print the winner. The constructor (RankedMatch.cpp, lines 4-12) calls the base Match constructor and determines the winner by finding the highest-MMR player. The Matchmaker stores matches as vector<shared_ptr<Match>> (Matchmaker.h, line 22) and calls match->display() (Matchmaker.cpp, line 128), using polymorphism to show the correct output.

7. Work Distribution

- Hoang Minh Tam Dang: 33.3% - Player class (Player.h, Player.cpp)
- Phu Vinh Khang Dang: 33.3% - Match and RankedMatch classes (Match.h/cpp, RankedMatch.h/cpp)
- Anh Tuan Huynh: 33.3% - Matchmaker, Queue, and main program (Matchmaker.h/cpp, Queue.h, main.cpp)

8. Academic Integrity Statement

We understand that there will be no tolerance towards academic dishonesty, and that cheating will lead to an academic referral. We are aware of the identified behaviors that are considered violations of the academic standards for Undergraduate and Graduate students per USF policy.